RAG-Based Customer Support Assistant

---

1. Purpose

This document defines the detailed design and execution model of the RAG-based Customer Support Assistant. It specifies component responsibilities, interaction contracts, failure handling, and runtime behavior required for production-grade implementation.

---

2. System Boundary Definition

In-Scope

- Query processing and response generation
- Document ingestion and indexing
- Retrieval and LLM orchestration
- Decision routing and escalation handling

External Dependencies

- LLM API (xAI Grok)
- Embedding model
- Vector database (ChromaDB)

Out-of-Scope

- Authentication and authorization
- Multi-tenant isolation
- Distributed deployment

---

3. Execution Model

The system follows a **synchronous request-response pipeline** with controlled branching:

Request → Validate → Retrieve → Generate → Decide → Respond | Escalate

Each stage is isolated and communicates via explicit contracts.

---

4. Component Interaction Model

Client

↓

API Layer

↓

Application Service

↓

Embedding Service

↓

Vector Store (Retrieval)

↓

LLM Service

↓

Decision Engine

↓

Response / Escalation Store

Interaction Characteristics

- All interactions are **synchronous**
- No shared mutable state across requests
- Each request is processed independently

- External services (LLM, embeddings) are treated as **unreliable dependencies**

---

5. Component Design

5.1 API Layer

Responsibilities

- Validate request schema
- Enforce input constraints
- Return standardized responses

Request Contract

```
{
  "question": "string",
  "knowledge_base_id": "string",
  "top_k": integer
}
```

Response Contract

```
{
  "status": "success | error",
  "route": "answer | escalate",
  "answer": "string | null",
  "confidence": float,
  "ticket_id": "string | null",
  "error": "string | null"
}
```

5.2 Application Service

Responsibilities

- Orchestrate execution pipeline
- Maintain request-scoped state
- Invoke downstream modules in sequence

Behavior

- Stateless execution
- No persistence at this layer
- Delegates all domain operations

5.3 Document Processing Module

Responsibilities

- Extract structured text from PDF

Output

[(page_number: int, content: str)]

5.4 Chunking Module

Responsibilities

- Convert text into retrievable units

Rules

- fixed chunk size
- overlapping window

- deterministic indexing

Structure

```
TextChunk(
    id: str,
    text: str,
    page_number: int,
    index: int
)
```

5.5 Embedding Module

Responsibilities

- Generate vector representation

Operational Constraints

- batch processing for ingestion
- single-call inference for query

5.6 Vector Store Module

Responsibilities

- Persist embeddings
- Execute similarity search

Query Behavior

- top-K nearest neighbor search
- cosine similarity scoring

5.7 Retrieval Module

Responsibilities

- Fetch relevant context

Execution

Query → Embed → Search → Rank → Return Top-K

Output

```
RetrievedChunk(
    id: str,
    text: str,
    score: float,
    metadata: dict
)
```

5.8 LLM Service

Responsibilities

- Generate grounded response

Input

- user query
- retrieved context

Output Contract

```
{
```

```
  "answer": "string",
  "confidence": float,
  "citations": [],
  "escalate": boolean,
  "reason": "string"
}
```

Constraints

- must not generate outside context
- must return structured output

5.9 Decision Engine

Responsibilities

- Determine system outcome

Inputs

- retrieval_score
- llm_confidence
- escalation_flag

Routing Logic

IF no_chunks → escalate

ELSE IF retrieval_score < threshold → escalate

ELSE IF llm_confidence < threshold → escalate

ELSE IF escalate_flag = true → escalate

ELSE → answer

5.10 Escalation Module

Responsibilities

- Persist unresolved queries
- Enable human resolution

Storage

- SQLite

Ticket Model

```
{
  "id": "uuid",
  "query": "string",
  "reason": "string",
  "status": "open | resolved",
  "draft_answer": "string",
  "context": [],
  "created_at": "timestamp",
  "resolved_at": "timestamp"
}
```

---

6. End-to-End Execution Flow

1. Receive request
2. Validate input
3. Generate query embedding
4. Retrieve top-K chunks

5. Compute retrieval score

6. Invoke LLM

7. Parse structured response

8. Apply decision logic

9. Return response OR create escalation

---

7. Failure Handling Design

Failure Categories

1. Retrieval Failure

- no relevant chunks found
  → route to escalation

---

2. LLM Failure

- timeout / API error
  → retry (limited) → escalate

---

3. Parsing Failure

- invalid LLM response
  → escalate

---

4. Dependency Failure

- embedding or DB failure
  → fail fast with error response

Retry Policy

- LLM: max 1 retry

- retrieval: no retry
- ingestion: fail immediately

---

8. Operational Considerations

Latency Breakdown

- embedding: moderate
- retrieval: low
- LLM: high (dominant factor)

Bottlenecks

- LLM response time
- large prompt size

Mitigation

- limit top-K
- optimize chunk size
- enforce token limits

---

9. Observability

Captured for each request:

- query input
- retrieved chunks
- similarity scores
- LLM output
- decision outcome

---

10. Extensibility Design

The system supports replacement of:

- embedding model
- vector database
- LLM provider

via modular abstraction at service level.

---

11. Constraints

- stateless execution
- single knowledge base
- dependency on external APIs

---

12. Summary

This design defines a controlled execution architecture where:

- retrieval ensures context grounding
- generation is constrained and structured
- decision logic governs system behavior
- escalation guarantees safe fallback

The system prioritizes correctness and control over unrestricted automation.